Практическая работа №1

Развертывание контейнеризированных сред для нескольких типов баз данных

---

1. Цель работы

Целью данной практической работы является приобретение практических навыков развертывания и первоначальной настройки различных систем управления базами данных (СУБД) с использованием технологии контейнеризации Docker. В ходе работы студенты:

- познакомятся с основными типами современных СУБД (реляционные, документоориентированные, графовые, поисковые, key-value);
- освоят создание конфигурационных файлов Docker Compose для развертывания каждой СУБД;
- научатся подключаться к каждой СУБД через командную строку (CLI) и веб-интерфейс;
- выполнят базовые операции с данными в каждой из систем.

---

2. Теоретическая часть

2.1. Что такое Docker и Docker Compose

Docker --- это платформа для разработки, доставки и запуска приложений в изолированных контейнерах. Контейнер --- это легковесная, автономная исполняемая единица, которая включает в себя всё необходимое для работы приложения: код, среду выполнения, библиотеки, системные инструменты и настройки.

Docker Compose --- инструмент для определения и запуска многоконтейнерных Docker-приложений. С помощью файла `docker-compose.yml` вы описываете сервисы, сети и тома, а затем одной командой запускаете всё окружение.

Основные понятия:

Понятие	Описание
Image (образ)	Шаблон, на основе которого создается контейнер. Содержит ОС, ПО и настройки
Container (контейнер)	Запущенный экземпляр образа. Изолированная среда выполнения
Volume (том)	Механизм хранения данных вне контейнера. Данные сохраняются после удаления контейнера
Port mapping (проброс портов)	Связывание порта хост-машины с портом контейнера для доступа к сервисам
Network (сеть)	Виртуальная сеть для общения контейнеров между собой
Environment variables	Переменные окружения для настройки сервиса внутри контейнера

2.2. Обзор СУБД, используемых в работе

2.2.1. PostgreSQL (реляционная СУБД)

PostgreSQL --- мощная объектно-реляционная система управления базами данных с открытым исходным кодом. Разрабатывается более 35 лет и зарекомендовала себя как надежная, функциональная и производительная СУБД.

- **Модель данных**: реляционная (таблицы, строки, столбцы)
- **Язык запросов**: SQL (с расширениями)
- **Применение**: веб-приложения, финансовые системы, аналитика, геоинформационные системы (через PostGIS)
- **Ключевые особенности**: ACID-транзакции, поддержка JSON/JSONB, полнотекстовый поиск, расширяемость, репликация

2.2.2. MongoDB (документоориентированная СУБД)

MongoDB --- документоориентированная NoSQL база данных, хранящая данные в формате BSON (Binary JSON).

- **Модель данных**: документная (коллекции документов в формате JSON/BSON)
- **Язык запросов**: MQL (MongoDB Query Language)

- **Применение**: веб-приложения, мобильные приложения, IoT, системы управления контентом
- **Ключевые особенности**: гибкая схема данных, горизонтальное масштабирование (шардирование), встроенная репликация, агрегационный конвейер

2.2.3. Redis (key-value хранилище)

Redis (Remote Dictionary Server) --- высокопроизводительное хранилище данных типа «ключ-значение», работающее в оперативной памяти.

- **Модель данных**: ключ-значение (с поддержкой сложных структур: списки, множества, хеши, отсортированные множества, потоки)
- **Язык запросов**: собственный набор команд Redis
- **Применение**: кэширование, очереди сообщений, сессии пользователей, рейтинги в реальном времени, pub/sub
- **Ключевые особенности**: работа в оперативной памяти (скорость), поддержка персистентности (RDB/AOF), pub/sub, Lua-скрипты, кластеризация

2.2.4. Neo4j (графовая СУБД)

Neo4j --- ведущая графовая база данных, хранящая данные в виде узлов и связей (ребер) между ними.

- **Модель данных**: графовая (узлы, связи, свойства)
- **Язык запросов**: Cypher
- **Применение**: социальные сети, системы рекомендаций, анализ мошенничества, управление знаниями, сетевой анализ
- **Ключевые особенности**: нативное графовое хранение, язык Cypher, встроенный веб-интерфейс Neo4j Browser, ACID-транзакции

2.2.5. Elasticsearch (поисковая система)

Elasticsearch --- распределенная поисковая и аналитическая система, основанная на Apache Lucene.

- **Модель данных**: документная (индексы, содержащие JSON-документы)
- **Язык запросов**: Query DSL (JSON-based), SQL-подобный синтаксис
- **Применение**: полнотекстовый поиск, логирование и мониторинг (ELK-стек), аналитика в реальном времени

- **Ключевые особенности**: полнотекстовый поиск, распределенная архитектура, REST API, агрегации, near real-time поиск

2.3. Сравнительная таблица СУБД

Характеристика	PostgreSQL	MongoDB	Redis	Neo4j	
Тип	Реляционная	Документная	Key-Value	Графовая	
Формат данных	Таблицы	JSON/BSON	Ключ-Значение	Граф (узлы/ребра)	
Язык запросов	SQL	MQL	Команды Redis	Cypher	
Порт по умолчанию	5432	27017	6379	7474 (HTTP), 7687 (Bolt)	
ACID	Да	Да (с v4.0)	Частично	Да	
Основное применение	Общего назначения	Гибкие данные	Кэш, очереди	Связанные данные	

---

3. Предварительные требования

3.1. Установка Docker Desktop

1. Перейдите на официальный сайт Docker: https://www.docker.com/products/docker-desktop
2. Скачайте Docker Desktop для вашей операционной системы (Windows / macOS / Linux).
3. Установите Docker Desktop, следуя инструкциям установщика.
4. После установки запустите Docker Desktop и дождитесь, пока он полностью загрузится (иконка в системном трее перестанет анимироваться).

3.2. Проверка установки

Откройте терминал (PowerShell, cmd или bash) и выполните следующие команды:

```
# Проверка версии Docker
docker --version
# Ожидаемый вывод: Docker version 27.x.x, build xxxxxxx

# Проверка версии Docker Compose
docker compose version
# Ожидаемый вывод: Docker Compose version v2.x.x

# Проверка, что Docker-демон запущен
docker info
# Должна отобразиться информация о системе Docker
```

3.3. Подготовка рабочей директории

Создайте рабочую директорию для практической работы:

```
mkdir -p ~/practice01-databases
cd ~/practice01-databases
```

Внутри этой директории мы будем создавать подкаталоги для каждой СУБД.

---

4. Пошаговое выполнение

4.1. Redis

4.1.1. Описание конфигурации

Создайте директорию и файл конфигурации:

```
mkdir -p ~/practice01-databases/redis
cd ~/practice01-databases/redis
```

Создайте файл `docker-compose.yml` со следующим содержимым:

```
# Файл: redis/docker-compose.yml
# Описание: Развертывание Redis с веб-интерфейсом Redis
Commander

# Версия формата Docker Compose файла (необязательно для Docker
Compose V2,
# но указывается для совместимости)
version: "3.9"

services:

  # === Сервис 1: Redis-сервер ===
  redis:
    # Образ Redis из Docker Hub. Тег "7-alpine" означает:
    #   7       — мажорная версия Redis
    #   alpine  — легковесный образ на базе Alpine Linux (~5 MB
вместо ~100 MB)
    image: redis:7-alpine

    # Имя контейнера для удобства идентификации
    # (по умолчанию Docker генерирует случайное имя)
    container_name: practice01-redis

    # Политика перезапуска контейнера:
    #   "unless-stopped" — контейнер перезапускается
автоматически,
    #   если он был остановлен не вручную (например, при
перезагрузке хоста)
    restart: unless-stopped

    # Проброс портов: <порт_хоста>:<порт_контейнера>
    # 6379 — стандартный порт Redis
    # После запуска Redis будет доступен на localhost:6379
    ports:
      - "6379:6379"

    # Тома (volumes) для постоянного хранения данных.
    # Без этой настройки данные пропадут после удаления
контейнера.
    # "redis_data" — именованный том Docker, монтируется в /data
внутри контейнера,
    # где Redis сохраняет дампы (RDB) и AOF-файлы.
    volumes:
```

```
      - redis_data:/data

    # Команда запуска Redis-сервера с дополнительными
параметрами:
    #   --requirepass mypassword  — установить пароль для
подключения
    #   --maxmemory 256mb         — ограничить использование
памяти (256 МБ)
    #   --maxmemory-policy allkeys-lru — при превышении лимита
памяти
    #                                    удалять наименее
используемые ключи
    command: >
      redis-server
      --requirepass mypassword
      --maxmemory 256mb
      --maxmemory-policy allkeys-lru

    # Проверка здоровья (healthcheck) — Docker будет
периодически проверять,
    # что Redis отвечает на команду PING.
    healthcheck:
      # Команда проверки: redis-cli подключается с паролем и
отправляет PING
      test: ["CMD", "redis-cli", "-a", "mypassword", "ping"]
      # Интервал между проверками
      interval: 10s
      # Максимальное время ожидания ответа
      timeout: 5s
      # Количество неудачных попыток, после которых контейнер
считается unhealthy
      retries: 3

  # === Сервис 2: Redis Commander (веб-интерфейс) ===
  redis-commander:
    # Образ Redis Commander — веб-клиент для управления Redis
    image: rediscommander/redis-commander:latest

    container_name: practice01-redis-commander

    restart: unless-stopped

    # Переменные окружения для настройки Redis Commander:
    environment:
      # Хост Redis-сервера. Используется имя сервиса из docker-
compose (redis),
```

```
      # так как контейнеры находятся в одной Docker-сети
      - REDIS_HOSTS=local:redis:6379:0:mypassword

    # Веб-интерфейс будет доступен на http://localhost:8081
    ports:
      - "8081:8081"

    # Этот сервис запустится только после того, как Redis будет
готов (healthy)
    depends_on:
      redis:
        condition: service_healthy

# Определение именованных томов.
# Docker создаст том "redis_data" и будет хранить данные
# в специальной директории на хост-машине.
volumes:
  redis_data:
    # Драйвер хранения. "local" — стандартный, хранит данные
    # на файловой системе хоста
    driver: local
```

4.1.2. Запуск

```
cd ~/practice01-databases/redis

# Запуск всех сервисов в фоновом режиме (-d = detached)
docker compose up -d

# Проверка статуса контейнеров
docker compose ps
```

Ожидаемый вывод:

```
NAME                          STATUS                    PORTS
practice01-redis              Up (healthy)              0.0.0.0:6379-
>6379/tcp
practice01-redis-commander    Up                        0.0.0.0:8081-
>8081/tcp
```

4.1.3. Проверка работоспособности

```
# Просмотр логов Redis
docker compose logs redis

# Проверка, что Redis отвечает
docker exec practice01-redis redis-cli -a mypassword ping
# Ожидаемый ответ: PONG
```

4.1.4. Подключение через CLI

```
# Подключение к Redis через CLI внутри контейнера
docker exec -it practice01-redis redis-cli -a mypassword
```

После подключения вы увидите приглашение `127.0.0.1:6379>`. Выполните базовые команды:

```
# === Работа со строками ===

# SET — сохранить значение по ключу
SET mykey "Hello, Redis!"
# Ответ: OK

# GET — получить значение по ключу
GET mykey
# Ответ: "Hello, Redis!"

# SET с TTL (время жизни ключа в секундах)
SET session:user1 "token123" EX 3600
# Ключ автоматически удалится через 3600 секунд (1 час)

# TTL — узнать оставшееся время жизни ключа
TTL session:user1
# Ответ: число секунд до удаления

# === Работа с хешами (аналог объектов/словарей) ===

# HSET — сохранить поля хеша
HSET user:1 name "Ivanov" age "20" group "IKBO-01-22"
# Ответ: (integer) 3
```

```
# HGETALL — получить все поля и значения хеша
HGETALL user:1
# Ответ:
# 1) "name"
# 2) "Ivanov"
# 3) "age"
# 4) "20"
# 5) "group"
# 6) "IKBO-01-22"

# HGET — получить одно поле
HGET user:1 name
# Ответ: "Ivanov"

# === Работа со списками ===

# LPUSH — добавить элемент в начало списка
LPUSH tasks "task1" "task2" "task3"

# LRANGE — получить элементы списка (0 = первый, -1 = последний)
LRANGE tasks 0 -1
# Ответ: 1) "task3"  2) "task2"  3) "task1"

# === Работа с множествами ===

# SADD — добавить элементы в множество
SADD languages "Python" "Java" "Go" "Python"
# "Python" добавится только один раз (множество не допускает
дубликатов)

# SMEMBERS — получить все элементы множества
SMEMBERS languages

# === Служебные команды ===

# KEYS * — показать все ключи (не использовать на production!)
KEYS *

# DBSIZE — количество ключей в текущей базе
DBSIZE

# INFO — информация о сервере
INFO server
```

```
# Выход из CLI
EXIT
```

4.1.5. Подключение через веб-интерфейс

1. Откройте в браузере: **http://localhost:8081**
2. Вы увидите интерфейс Redis Commander с древовидной структурой ключей.
3. Здесь можно просматривать, создавать, редактировать и удалять ключи.
4. В левой панели отображается список баз данных (db0-db15) и ключи.
5. Попробуйте найти ключи, созданные ранее через CLI (`mykey` , `user:1` , `tasks` , `languages`).

4.2. MongoDB

4.2.1. Описание конфигурации

```
mkdir -p ~/practice01-databases/mongodb
cd ~/practice01-databases/mongodb
```

Создайте файл `docker-compose.yml` :

```
# Файл: mongodb/docker-compose.yml
# Описание: Развертывание MongoDB с веб-интерфейсом Mongo
Express

version: "3.9"

services:

  # === Сервис 1: MongoDB-сервер ===
  mongodb:
    # Образ MongoDB версии 7.
    # MongoDB — документоориентированная NoSQL СУБД.
    image: mongo:7

    container_name: practice01-mongodb
```

```
    restart: unless-stopped

    # Переменные окружения для инициализации MongoDB:
    environment:
      # Имя пользователя root (администратор).
      # MongoDB создаст этого пользователя при первом запуске.
      MONGO_INITDB_ROOT_USERNAME: admin
      # Пароль пользователя root
      MONGO_INITDB_ROOT_PASSWORD: secret
      # Имя базы данных, которая будет создана при инициализации
      MONGO_INITDB_DATABASE: practice_db

    # Проброс порта: 27017 — стандартный порт MongoDB
    ports:
      - "27017:27017"

    # Том для хранения данных MongoDB.
    # /data/db — стандартная директория хранения данных MongoDB
внутри контейнера.
    volumes:
      - mongodb_data:/data/db

    # Дополнительные параметры запуска MongoDB:
    #   --wiredTigerCacheSizeGB 0.5 — ограничить кеш движка
WiredTiger до 0.5 ГБ
    #                                 (по умолчанию MongoDB
может забрать до 50% RAM)
    command: mongod --wiredTigerCacheSizeGB 0.5

    # Проверка здоровья: mongosh выполняет команду
db.adminCommand('ping')
    healthcheck:
      test: ["CMD", "mongosh", "--eval",
"db.adminCommand('ping')", "-u", "admin", "-p", "secret", "--
authenticationDatabase", "admin", "--quiet"]
      interval: 10s
      timeout: 5s
      retries: 5

  # === Сервис 2: Mongo Express (веб-интерфейс) ===
  mongo-express:
    # Mongo Express — легковесный веб-клиент для MongoDB
    image: mongo-express:1-20-alpine3.19

    container_name: practice01-mongo-express
```

```
    restart: unless-stopped

    environment:
      # Учетные данные для подключения Mongo Express к MongoDB
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: secret
      # URL подключения к MongoDB.
      # "mongodb" — имя сервиса в docker-compose, Docker
автоматически
      # разрешает его в IP-адрес контейнера.
      ME_CONFIG_MONGODB_URL:
mongodb://admin:secret@mongodb:27017/
      # Учетные данные для входа в веб-интерфейс Mongo Express
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin

    # Веб-интерфейс доступен на http://localhost:8082
    ports:
      - "8082:8081"

    # Запускать только после готовности MongoDB
    depends_on:
      mongodb:
        condition: service_healthy

volumes:
  mongodb_data:
    driver: local
```

4.2.2. Запуск

```
cd ~/practice01-databases/mongodb

docker compose up -d
docker compose ps
```

4.2.3. Проверка работоспособности

```
# Просмотр логов
docker compose logs mongodb
```

```
# Быстрая проверка подключения
docker exec practice01-mongodb mongosh -u admin -p secret --
authenticationDatabase admin --eval "db.runCommand({ping: 1})"
# Ожидаемый ответ: { ok: 1 }
```

4.2.4. Подключение через CLI

```
# Подключение к MongoDB Shell (mongosh)
docker exec -it practice01-mongodb mongosh -u admin -p secret --
authenticationDatabase admin
```

После подключения выполните базовые операции:

```
// === Работа с базами данных ===

// Показать все базы данных
show dbs

// Переключиться на базу данных (создается автоматически при
первой записи)
use practice_db

// === Работа с коллекциями и документами ===

// Вставить один документ в коллекцию "students"
// (коллекция создается автоматически)
db.students.insertOne({
    name: "Иванов Иван",
    age: 20,
    group: "ИКБО-01-22",
    grades: {
        math: 5,
        physics: 4,
        programming: 5
    },
    hobbies: ["программирование", "шахматы"]
})
// Ответ: { acknowledged: true, insertedId: ObjectId("...") }

// Вставить несколько документов
```

```
db.students.insertMany([
    {
        name: "Петрова Мария",
        age: 21,
        group: "ИКБО-02-22",
        grades: { math: 4, physics: 5, programming: 4 },
        hobbies: ["рисование", "волейбол"]
    },
    {
        name: "Сидоров Алексей",
        age: 19,
        group: "ИКБО-01-22",
        grades: { math: 3, physics: 4, programming: 5 },
        hobbies: ["программирование", "музыка"]
    }
])

// === Чтение данных ===

// Найти все документы в коллекции
db.students.find()

// Найти с форматированным выводом
db.students.find().pretty()

// Найти студентов определенной группы
db.students.find({ group: "ИКБО-01-22" })

// Найти студентов старше 19 лет
db.students.find({ age: { $gt: 19 } })

// Найти студентов с оценкой 5 по программированию
db.students.find({ "grades.programming": 5 })

// Подсчитать количество документов
db.students.countDocuments()

// === Обновление данных ===

// Обновить один документ
db.students.updateOne(
    { name: "Иванов Иван" },           // фильтр
    { $set: { age: 21 } }              // обновление
)

// Добавить хобби в массив
```

```
db.students.updateOne(
    { name: "Иванов Иван" },
    { $push: { hobbies: "фотография" } }
)

// === Удаление данных ===

// Удалить один документ
// db.students.deleteOne({ name: "Сидоров Алексей" })

// === Информация о коллекции ===

// Показать все коллекции в текущей БД
show collections

// Статистика коллекции
db.students.stats()

// Выход
exit
```

4.2.5. Подключение через веб-интерфейс

1. Откройте в браузере: **http://localhost:8082**
2. Введите логин/пароль: `admin` / `admin`
3. Вы увидите список баз данных. Выберите `practice_db`.
4. Перейдите в коллекцию `students` --- вы увидите все добавленные документы.
5. Через интерфейс можно создавать/редактировать/удалять базы данных, коллекции и документы.

4.3. Neo4j

4.3.1. Описание конфигурации

```
mkdir -p ~/practice01-databases/neo4j
cd ~/practice01-databases/neo4j
```

Создайте файл `docker-compose.yml` :

```
# Файл: neo4j/docker-compose.yml
# Описание: Развертывание Neo4j — графовой базы данных
# Neo4j имеет ВСТРОЕННЫЙ веб-интерфейс (Neo4j Browser), поэтому
# дополнительный веб-клиент не требуется.

version: "3.9"

services:

  neo4j:
    # Образ Neo4j Community Edition версии 5
    # Community Edition — бесплатная версия с ограничениями
    # (одна БД, без кластеризации)
    image: neo4j:5-community

    container_name: practice01-neo4j

    restart: unless-stopped

    # Проброс портов:
    ports:
      # 7474 — HTTP-порт для веб-интерфейса Neo4j Browser
      # Это встроенный веб-клиент для выполнения Cypher-запросов
      - "7474:7474"
      # 7687 — порт протокола Bolt (бинарный протокол для
подключения
      # драйверов и приложений к Neo4j)
      - "7687:7687"

    environment:
      # Начальный пароль для пользователя neo4j.
      # По умолчанию логин: neo4j
      NEO4J_AUTH: neo4j/mysecretpassword

      # Настройки памяти для JVM (Neo4j написан на Java):
      # Размер heap-памяти (начальный и максимальный).
      # Heap используется для выполнения запросов и транзакций.
      NEO4J_server_memory_heap_initial__size: 256m
      NEO4J_server_memory_heap_max__size: 512m

      # Размер page cache — кеш для хранения данных графа в
памяти.
      # Чем больше кеш, тем быстрее чтение данных из графа.
```

```
      NEO4J_server_memory_pagecache_size: 256m

      # Разрешить импорт файлов из любой директории
      # (по умолчанию только из /var/lib/neo4j/import)
      NEO4J_dbms_security_procedures_unrestricted: "apoc.*"

    volumes:
      # Данные графа (узлы, связи, свойства)
      - neo4j_data:/data
      # Логи Neo4j
      - neo4j_logs:/logs
      # Плагины (например, APOC)
      - neo4j_plugins:/plugins

    healthcheck:
      # Проверка через HTTP-эндпоинт Neo4j
      test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider
http://localhost:7474 || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 5

volumes:
  neo4j_data:
    driver: local
  neo4j_logs:
    driver: local
  neo4j_plugins:
    driver: local
```

4.3.2. Запуск

```
cd ~/practice01-databases/neo4j

docker compose up -d
docker compose ps
```

> ***Примечание***: *Neo4j может запускаться 30--60 секунд. Дождитесь статуса* `healthy` .

4.3.3. Проверка работоспособности

```
# Просмотр логов (дождитесь сообщения "Started.")
docker compose logs -f neo4j

# Проверка HTTP-интерфейса
curl http://localhost:7474
```

4.3.4. Подключение через CLI

```
# Подключение через cypher-shell внутри контейнера
docker exec -it practice01-neo4j cypher-shell -u neo4j -p
mysecretpassword
```

Выполните базовые запросы на языке **Cypher**:

```
// === Создание узлов ===

// Создать узел с меткой "Student" и свойствами
CREATE (s:Student {name: "Иванов Иван", age: 20, group:
"ИКБО-01-22"})
RETURN s;

// Создать несколько узлов
CREATE (s1:Student {name: "Петрова Мария", age: 21, group:
"ИКБО-02-22"}),
       (s2:Student {name: "Сидоров Алексей", age: 19, group:
"ИКБО-01-22"}),
       (c1:Course {title: "Базы данных", credits: 4}),
       (c2:Course {title: "Программирование", credits: 5}),
       (c3:Course {title: "Математика", credits: 6});

// === Создание связей ===

// Связать студентов с курсами (отношение ENROLLED_IN)
MATCH (s:Student {name: "Иванов Иван"}), (c:Course {title: "Базы
данных"})
CREATE (s)-[:ENROLLED_IN {grade: 5, semester: "весна 2024"}]->
(c)
```

```
RETURN s, c;

MATCH (s:Student {name: "Иванов Иван"}), (c:Course {title:
"Программирование"})
CREATE (s)-[:ENROLLED_IN {grade: 5, semester: "весна 2024"}]->
(c);

MATCH (s:Student {name: "Петрова Мария"}), (c:Course {title:
"Базы данных"})
CREATE (s)-[:ENROLLED_IN {grade: 4, semester: "весна 2024"}]->
(c);

MATCH (s:Student {name: "Петрова Мария"}), (c:Course {title:
"Математика"})
CREATE (s)-[:ENROLLED_IN {grade: 5, semester: "весна 2024"}]->
(c);

// Связь "дружбы" между студентами
MATCH (s1:Student {name: "Иванов Иван"}), (s2:Student {name:
"Сидоров Алексей"})
CREATE (s1)-[:FRIENDS_WITH {since: "2022-09-01"}]->(s2);

// === Чтение данных ===

// Получить всех студентов
MATCH (s:Student) RETURN s;

// Получить все узлы и связи
MATCH (n)-[r]->(m) RETURN n, r, m;

// Найти все курсы студента "Иванов Иван"
MATCH (s:Student {name: "Иванов Иван"})-[r:ENROLLED_IN]->
(c:Course)
RETURN c.title AS course, r.grade AS grade;

// Найти всех студентов на курсе "Базы данных"
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {title: "Базы
данных"})
RETURN s.name AS student, s.group AS studentGroup;

// Найти друзей друзей (обход графа в глубину)
MATCH (s:Student {name: "Иванов Иван"})-[:FRIENDS_WITH*1..2]-
(friend:Student)
RETURN friend.name AS friendName;

// Подсчитать количество студентов на каждом курсе
```

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
RETURN c.title AS course, count(s) AS studentCount
ORDER BY studentCount DESC;

// === Удаление данных ===
// (раскомментируйте при необходимости)
// MATCH (n) DETACH DELETE n;  // Удалить ВСЕ данные из базы

// Выход
:exit
```

4.3.5. Подключение через веб-интерфейс

1. Откройте в браузере: **http://localhost:7474**
2. Вы увидите **Neo4j Browser** --- встроенный веб-интерфейс.
3. Подключитесь с логином `neo4j` и паролем `mysecretpassword`.
4. В верхней строке введите Cypher-запрос, например:

   ```
   MATCH (n)-[r]->(m) RETURN n, r, m
   ```

5. Нажмите **Play** (или Ctrl+Enter). Результат отобразится в виде интерактивного графа.
6. Вы можете переключаться между режимами отображения: граф, таблица, текст.

> ***Совет**: Neo4j Browser --- один из лучших встроенных веб-интерфейсов среди СУБД. Он позволяет визуализировать граф, выполнять запросы, просматривать схему данных и изучать синтаксис Cypher через встроенные туториалы (введите `:play start`).*

4.4. Elasticsearch

4.4.1. Описание конфигурации

```
mkdir -p ~/practice01-databases/elasticsearch
cd ~/practice01-databases/elasticsearch
```

Создайте файл `docker-compose.yml` :

```
# Файл: elasticsearch/docker-compose.yml
# Описание: Развертывание Elasticsearch с веб-интерфейсом Kibana

version: "3.9"

services:

  # === Сервис 1: Elasticsearch ===
  elasticsearch:
    # Образ Elasticsearch версии 8.
    # Elasticsearch — распределенная поисковая и аналитическая
система.
    image: elasticsearch:8.16.0

    container_name: practice01-elasticsearch

    restart: unless-stopped

    environment:
      # Имя узла (ноды) кластера Elasticsearch.
      # Каждый экземпляр Elasticsearch в кластере должен иметь
уникальное имя.
      - node.name=es-node-1

      # Имя кластера. Все ноды с одинаковым именем кластера
      # объединяются в один кластер.
      - cluster.name=practice-cluster

      # Режим одного узла (single-node).
      # Для учебных целей используем один узел без репликации.
      # В production обычно используется кластер из 3+ узлов.
      - discovery.type=single-node

      # Отключаем SSL/TLS (для упрощения в учебных целях).
      # В production ОБЯЗАТЕЛЬНО включать!
      - xpack.security.enabled=false

      # Отключаем enrollment token (новая фича v8)
      - xpack.security.enrollment.enabled=false

      # Ограничение Java heap (Elasticsearch написан на Java).
      # ES_JAVA_OPTS задает параметры JVM.
      # -Xms512m — начальный размер heap
```

```
      # -Xmx512m — максимальный размер heap
      # Рекомендуется устанавливать оба значения одинаковыми.
      - ES_JAVA_OPTS=-Xms512m -Xmx512m

      # Отключаем Machine Learning (требует много ресурсов)
      - xpack.ml.enabled=false

    ports:
      # 9200 — HTTP REST API (основной порт для запросов)
      - "9200:9200"
      # 9300 — порт для межузлового взаимодействия (transport).
      # Используется для общения между нодами кластера.
      - "9300:9300"

    volumes:
      # Данные индексов Elasticsearch
      - es_data:/usr/share/elasticsearch/data

    # Ограничение ресурсов (Elasticsearch может потреблять много
памяти)
    deploy:
      resources:
        limits:
          memory: 1G

    # Увеличиваем лимит на количество mmap-областей памяти.
    # Elasticsearch использует mmap для работы с файлами
индексов.
    # Без этой настройки ES может не запуститься.
    ulimits:
      memlock:
        soft: -1
        hard: -1

    healthcheck:
      test: ["CMD-SHELL", "curl -f
http://localhost:9200/_cluster/health || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 5

  # === Сервис 2: Kibana (веб-интерфейс) ===
  kibana:
    # Kibana — мощный веб-интерфейс для Elasticsearch.
    # Позволяет визуализировать данные, выполнять запросы,
    # строить дашборды и многое другое.
```

```
    image: kibana:8.16.0

    container_name: practice01-kibana

    restart: unless-stopped

    environment:
      # URL для подключения к Elasticsearch.
      # Используем имя сервиса из docker-compose.
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200

    # Веб-интерфейс Kibana доступен на http://localhost:5601
    ports:
      - "5601:5601"

    depends_on:
      elasticsearch:
        condition: service_healthy

    deploy:
      resources:
        limits:
          memory: 1G

volumes:
  es_data:
    driver: local
```

4.4.2. Запуск

```
cd ~/practice01-databases/elasticsearch

docker compose up -d
docker compose ps
```

> ***Внимание***: *Elasticsearch и Kibana --- ресурсоемкие приложения. Убедитесь, что на вашем компьютере достаточно оперативной памяти (минимум 4 ГБ свободных). Полная загрузка может занять 1--3 минуты.*

4.4.3. Проверка работоспособности

```
# Проверка через curl (REST API)
curl http://localhost:9200
# Ожидаемый ответ: JSON с информацией о кластере

# Проверка здоровья кластера
curl http://localhost:9200/_cluster/health?pretty
# Статус "green" или "yellow" означает, что кластер работает
```

4.4.4. Подключение через CLI (REST API с curl)

Elasticsearch не имеет собственного CLI-клиента. Взаимодействие осуществляется через **REST API** с помощью утилиты `curl`. Все команды выполняются на хост-машине:

```
# === Создание индекса ===
# Индекс в Elasticsearch — аналог базы данных/таблицы.
# PUT /имя_индекса — создать индекс с настройками.
curl -X PUT "http://localhost:9200/students" -H "Content-Type:
application/json" -d '{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0
  },
  "mappings": {
    "properties": {
      "name": { "type": "text", "analyzer": "standard" },
      "age": { "type": "integer" },
      "group": { "type": "keyword" },
      "bio": { "type": "text" },
      "enrolled_at": { "type": "date" }
    }
  }
}'

# === Индексация документов (добавление данных) ===
# POST /имя_индекса/_doc — добавить документ с автоматическим ID
curl -X POST "http://localhost:9200/students/_doc" -H "Content-
Type: application/json" -d '{
  "name": "Иванов Иван Иванович",
  "age": 20,
  "group": "ИКБО-01-22",
  "bio": "Студент третьего курса, увлекается программированием и
```

```
машинным обучением",
  "enrolled_at": "2022-09-01"
}'

# Добавить документ с конкретным ID
curl -X PUT "http://localhost:9200/students/_doc/2" -H "Content-
Type: application/json" -d '{
  "name": "Петрова Мария Сергеевна",
  "age": 21,
  "group": "ИКБО-02-22",
  "bio": "Студентка, занимается анализом данных и
визуализацией",
  "enrolled_at": "2022-09-01"
}'

curl -X PUT "http://localhost:9200/students/_doc/3" -H "Content-
Type: application/json" -d '{
  "name": "Сидоров Алексей Петрович",
  "age": 19,
  "group": "ИКБО-01-22",
  "bio": "Первокурсник, интересуется веб-разработкой и базами
данных",
  "enrolled_at": "2023-09-01"
}'

# === Поиск данных ===

# Получить все документы из индекса
curl "http://localhost:9200/students/_search?pretty"

# Полнотекстовый поиск (главная сила Elasticsearch!)
# Найти студентов, в описании которых есть "программирование"
curl -X GET "http://localhost:9200/students/_search?pretty" -H
"Content-Type: application/json" -d '{
  "query": {
    "match": {
      "bio": "программирование"
    }
  }
}'

# Точное совпадение по ключевому полю (keyword)
curl -X GET "http://localhost:9200/students/_search?pretty" -H
"Content-Type: application/json" -d '{
  "query": {
    "term": {
```

```
      "group": "ИКБО-01-22"
    }
  }
}'

# Комбинированный запрос (bool query):
# Найти студентов группы ИКБО-01-22 старше 19 лет
curl -X GET "http://localhost:9200/students/_search?pretty" -H
"Content-Type: application/json" -d '{
  "query": {
    "bool": {
      "must": [
        { "term": { "group": "ИКБО-01-22" } },
        { "range": { "age": { "gt": 19 } } }
      ]
    }
  }
}'

# === Агрегации (аналитика) ===
# Подсчитать количество студентов в каждой группе
curl -X GET "http://localhost:9200/students/_search?pretty" -H
"Content-Type: application/json" -d '{
  "size": 0,
  "aggs": {
    "students_per_group": {
      "terms": {
        "field": "group"
      }
    }
  }
}'

# === Информация об индексах ===

# Список всех индексов
curl "http://localhost:9200/_cat/indices?v"

# Информация о маппинге (схеме) индекса
curl "http://localhost:9200/students/_mapping?pretty"

# Количество документов в индексе
curl "http://localhost:9200/students/_count?pretty"
```

4.4.5. Подключение через веб-интерфейс (Kibana)

1. Откройте в браузере: **http://localhost:5601**
2. Дождитесь полной загрузки Kibana (может занять 1--2 минуты при первом запуске).
3. Перейдите в раздел **Management** > **Dev Tools** (или используйте поиск в верхней панели).
4. В консоли Dev Tools можно выполнять запросы к Elasticsearch в упрощенном формате:

```
GET _cluster/health

GET students/_search
{
  "query": {
    "match_all": {}
  }
}

GET students/_search
{
  "query": {
    "match": {
      "bio": "программирование"
    }
  }
}
```

5. Для визуализации данных перейдите в **Analytics** > **Discover**. Создайте Data View для индекса `students` и исследуйте данные.

4.5. PostgreSQL

4.5.1. Описание конфигурации

```
mkdir -p ~/practice01-databases/postgresql
cd ~/practice01-databases/postgresql
```

Создайте файл `docker-compose.yml` :

```
# Файл: postgresql/docker-compose.yml
# Описание: Развертывание PostgreSQL с веб-интерфейсом pgAdmin 4

version: "3.9"

services:

  # === Сервис 1: PostgreSQL-сервер ===
  postgres:
    # Образ PostgreSQL версии 16 на базе Alpine Linux.
    # PostgreSQL — мощная объектно-реляционная СУБД с открытым
исходным кодом.
    image: postgres:16-alpine

    container_name: practice01-postgres

    restart: unless-stopped

    environment:
      # Имя суперпользователя (по умолчанию "postgres")
      POSTGRES_USER: admin
      # Пароль суперпользователя (ОБЯЗАТЕЛЬНЫЙ параметр)
      POSTGRES_PASSWORD: secret
      # Имя базы данных, создаваемой при инициализации
      # (по умолчанию совпадает с POSTGRES_USER)
      POSTGRES_DB: practice_db
      # Кодировка базы данных
      POSTGRES_INITDB_ARGS: "--encoding=UTF-8 --lc-collate=C --
lc-ctype=C"
      # Директория данных внутри контейнера
      PGDATA: /var/lib/postgresql/data/pgdata

    ports:
      # 5432 — стандартный порт PostgreSQL
      - "5432:5432"

    volumes:
      # Том для данных PostgreSQL
      - postgres_data:/var/lib/postgresql/data
      # Скрипты инициализации: все .sql и .sh файлы из этой
директории
      # будут выполнены при ПЕРВОМ запуске контейнера (в
алфавитном порядке).
```

```
      # Это удобно для создания начальной схемы БД.
      # - ./init-scripts:/docker-entrypoint-initdb.d

    # Параметры запуска PostgreSQL для оптимизации под учебные
цели:
    command: >
      postgres
      -c shared_buffers=128MB
      -c max_connections=100
      -c log_statement=all
      -c log_destination=stderr

    # shared_buffers=128MB  — размер буферного кеша (по
умолчанию 128KB,
    #                         рекомендуется 25% от RAM для
production)
    # max_connections=100   — максимальное количество
подключений
    # log_statement=all     — логировать ВСЕ SQL-запросы
(полезно для обучения)
    # log_destination=stderr — направлять логи в stderr (виден
через docker logs)

    healthcheck:
      # pg_isready — утилита для проверки готовности PostgreSQL
      test: ["CMD-SHELL", "pg_isready -U admin -d practice_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  # === Сервис 2: pgAdmin 4 (веб-интерфейс) ===
  pgadmin:
    # pgAdmin 4 — полнофункциональный веб-клиент для PostgreSQL.
    # Позволяет управлять базами данных, выполнять SQL-запросы,
    # просматривать структуру БД, импортировать/экспортировать
данные.
    image: dpage/pgadmin4:latest

    container_name: practice01-pgadmin

    restart: unless-stopped

    environment:
      # Email для входа в pgAdmin (используется как логин)
      PGADMIN_DEFAULT_EMAIL: admin@example.com
      # Пароль для входа в pgAdmin
```

```
      PGADMIN_DEFAULT_PASSWORD: admin
      # Отключить отправку статистики использования
      PGADMIN_CONFIG_ENHANCED_COOKIE_PROTECTION: "False"

    ports:
      # Веб-интерфейс pgAdmin доступен на http://localhost:8083
      - "8083:80"

    volumes:
      # Том для хранения настроек pgAdmin (сохраненные серверы и
т.д.)
      - pgadmin_data:/var/lib/pgadmin

    depends_on:
      postgres:
        condition: service_healthy

volumes:
  postgres_data:
    driver: local
  pgadmin_data:
    driver: local
```

4.5.2. Запуск

```
cd ~/practice01-databases/postgresql

docker compose up -d
docker compose ps
```

4.5.3. Проверка работоспособности

```
# Просмотр логов
docker compose logs postgres

# Проверка готовности
docker exec practice01-postgres pg_isready -U admin -d
practice_db
```

```
# Ожидаемый ответ: /var/run/postgresql:5432 - accepting
connections
```

4.5.4. Подключение через CLI (psql)

```
# Подключение к PostgreSQL через psql
docker exec -it practice01-postgres psql -U admin -d practice_db
```

Выполните базовые SQL-запросы:

```
-- === Информация о системе ===

-- Версия PostgreSQL
SELECT version();

-- Список баз данных
\l

-- Текущая база данных и пользователь
SELECT current_database(), current_user;

-- === Создание таблиц ===

-- Создание таблицы групп
CREATE TABLE groups (
    id SERIAL PRIMARY KEY,          -- SERIAL — автоинкрементное
целое число
    name VARCHAR(20) NOT NULL UNIQUE,  -- Название группы
    department VARCHAR(100),        -- Факультет
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Создание таблицы студентов
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    age INTEGER CHECK (age > 0 AND age < 150),  -- CHECK —
ограничение на значения
    email VARCHAR(100) UNIQUE,
    group_id INTEGER REFERENCES groups(id),      -- Внешний ключ
```

```
(FOREIGN KEY)
    enrolled_at DATE DEFAULT CURRENT_DATE
);

-- Создание таблицы оценок
CREATE TABLE grades (
    id SERIAL PRIMARY KEY,
    student_id INTEGER REFERENCES students(id) ON DELETE
CASCADE,
    subject VARCHAR(100) NOT NULL,
    grade INTEGER CHECK (grade >= 1 AND grade <= 5),
    exam_date DATE
);

-- === Вставка данных ===

-- Вставка групп
INSERT INTO groups (name, department) VALUES
    ('ИКБО-01-22', 'Институт информационных технологий'),
    ('ИКБО-02-22', 'Институт информационных технологий'),
    ('ИМБО-01-22', 'Институт математики');

-- Вставка студентов
INSERT INTO students (first_name, last_name, age, email,
group_id) VALUES
    ('Иван', 'Иванов', 20, 'ivanov@mirea.ru', 1),
    ('Мария', 'Петрова', 21, 'petrova@mirea.ru', 2),
    ('Алексей', 'Сидоров', 19, 'sidorov@mirea.ru', 1),
    ('Елена', 'Козлова', 20, 'kozlova@mirea.ru', 3);

-- Вставка оценок
INSERT INTO grades (student_id, subject, grade, exam_date)
VALUES
    (1, 'Базы данных', 5, '2024-01-15'),
    (1, 'Программирование', 5, '2024-01-20'),
    (1, 'Математика', 4, '2024-01-10'),
    (2, 'Базы данных', 4, '2024-01-15'),
    (2, 'Программирование', 4, '2024-01-20'),
    (3, 'Базы данных', 3, '2024-01-15'),
    (3, 'Программирование', 5, '2024-01-20'),
    (4, 'Математика', 5, '2024-01-10');

-- === Чтение данных ===

-- Все студенты
SELECT * FROM students;
```

```sql
-- Студенты с их группами (JOIN)
SELECT s.first_name, s.last_name, g.name AS group_name
FROM students s
JOIN groups g ON s.group_id = g.id;

-- Средний балл каждого студента
SELECT s.first_name, s.last_name, ROUND(AVG(gr.grade), 2) AS
avg_grade
FROM students s
JOIN grades gr ON s.id = gr.student_id
GROUP BY s.id, s.first_name, s.last_name
ORDER BY avg_grade DESC;

-- Студенты с оценкой 5 по "Базы данных"
SELECT s.first_name, s.last_name
FROM students s
JOIN grades gr ON s.id = gr.student_id
WHERE gr.subject = 'Базы данных' AND gr.grade = 5;

-- Количество студентов в каждой группе
SELECT g.name, COUNT(s.id) AS student_count
FROM groups g
LEFT JOIN students s ON g.id = s.group_id
GROUP BY g.name;

-- === Обновление данных ===

UPDATE students SET age = 21 WHERE first_name = 'Иван' AND
last_name = 'Иванов';

-- === Полезные мета-команды psql ===

-- Список таблиц в текущей базе
\dt

-- Структура конкретной таблицы
\d students

-- Список схем
\dn

-- История команд
\s
```

```
-- Выход
\q
```

4.5.5. Подключение через веб-интерфейс (pgAdmin 4)

1. Откройте в браузере: **http://localhost:8083**
2. Введите логин: `admin@example.com`, пароль: `admin`.
3. Добавьте сервер PostgreSQL:
    - Нажмите правой кнопкой на **Servers** > **Register** > **Server...**
    - Вкладка **General**: Name = `Practice PostgreSQL`
    - Вкладка **Connection**:
        - Host name/address: `postgres` (имя сервиса из docker-compose)
        - Port: `5432`
        - Maintenance database: `practice_db`
        - Username: `admin`
        - Password: `secret`
        - Поставьте галочку **Save password**
    - Нажмите **Save**
4. В дереве слева раскройте: Servers > Practice PostgreSQL > Databases > practice_db > Schemas > public > Tables.
5. Для выполнения SQL-запросов: правый клик на базе данных > **Query Tool**.

---

5. Управление контейнерами

5.1. Основные команды Docker Compose

Все команды выполняются из директории, содержащей `docker-compose.yml`:

```
# Запуск всех сервисов в фоновом режиме
docker compose up -d

# Остановка всех сервисов (контейнеры сохраняются)
```

```
docker compose stop

# Запуск ранее остановленных сервисов
docker compose start

# Перезапуск всех сервисов
docker compose restart

# Остановка и удаление контейнеров (тома сохраняются!)
docker compose down

# Остановка, удаление контейнеров И томов (ДАННЫЕ БУДУТ
ПОТЕРЯНЫ!)
docker compose down -v

# Просмотр статуса контейнеров
docker compose ps

# Просмотр логов всех сервисов
docker compose logs

# Просмотр логов конкретного сервиса в реальном времени
docker compose logs -f redis

# Выполнение команды внутри контейнера
docker compose exec <имя_сервиса> <команда>
```

5.2. Полезные команды Docker

```
# Список всех запущенных контейнеров
docker ps

# Список всех контейнеров (включая остановленные)
docker ps -a

# Список томов
docker volume ls

# Удаление неиспользуемых ресурсов (осторожно!)
docker system prune
```

```
# Статистика использования ресурсов
docker stats
```

---

6. Единый docker-compose файл (дополнительно)

При необходимости все пять СУБД можно развернуть одним файлом. Создайте директорию `~/practice01-databases/all-in-one/` и файл `docker-compose.yml`:

```
# Файл: all-in-one/docker-compose.yml
# Описание: Все 5 СУБД в одном файле

version: "3.9"

services:

  # --- Redis ---
  redis:
    image: redis:7-alpine
    container_name: practice01-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --requirepass mypassword --maxmemory
256mb --maxmemory-policy allkeys-lru
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "mypassword", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3

  redis-commander:
    image: rediscommander/redis-commander:latest
    container_name: practice01-redis-commander
    restart: unless-stopped
    environment:
      - REDIS_HOSTS=local:redis:6379:0:mypassword
    ports:
      - "8081:8081"
```

```
    depends_on:
      redis:
        condition: service_healthy

  # --- MongoDB ---
  mongodb:
    image: mongo:7
    container_name: practice01-mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: secret
      MONGO_INITDB_DATABASE: practice_db
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
    command: mongod --wiredTigerCacheSizeGB 0.5
    healthcheck:
      test: ["CMD", "mongosh", "--eval",
"db.adminCommand('ping')", "-u", "admin", "-p", "secret", "--
authenticationDatabase", "admin", "--quiet"]
      interval: 10s
      timeout: 5s
      retries: 5

  mongo-express:
    image: mongo-express:1-20-alpine3.19
    container_name: practice01-mongo-express
    restart: unless-stopped
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: secret
      ME_CONFIG_MONGODB_URL:
mongodb://admin:secret@mongodb:27017/
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin
    ports:
      - "8082:8081"
    depends_on:
      mongodb:
        condition: service_healthy

  # --- Neo4j ---
  neo4j:
    image: neo4j:5-community
```

```
    container_name: practice01-neo4j
    restart: unless-stopped
    ports:
      - "7474:7474"
      - "7687:7687"
    environment:
      NEO4J_AUTH: neo4j/mysecretpassword
      NEO4J_server_memory_heap_initial__size: 256m
      NEO4J_server_memory_heap_max__size: 512m
      NEO4J_server_memory_pagecache_size: 256m
    volumes:
      - neo4j_data:/data
      - neo4j_logs:/logs
    healthcheck:
      test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider
http://localhost:7474 || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 5

  # --- Elasticsearch ---
  elasticsearch:
    image: elasticsearch:8.16.0
    container_name: practice01-elasticsearch
    restart: unless-stopped
    environment:
      - node.name=es-node-1
      - cluster.name=practice-cluster
      - discovery.type=single-node
      - xpack.security.enabled=false
      - xpack.security.enrollment.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
      - xpack.ml.enabled=false
    ports:
      - "9200:9200"
    volumes:
      - es_data:/usr/share/elasticsearch/data
    deploy:
      resources:
        limits:
          memory: 1G
    ulimits:
      memlock:
        soft: -1
        hard: -1
    healthcheck:
```

```
      test: ["CMD-SHELL", "curl -f
http://localhost:9200/_cluster/health || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 5

  kibana:
    image: kibana:8.16.0
    container_name: practice01-kibana
    restart: unless-stopped
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    ports:
      - "5601:5601"
    depends_on:
      elasticsearch:
        condition: service_healthy
    deploy:
      resources:
        limits:
          memory: 1G

  # --- PostgreSQL ---
  postgres:
    image: postgres:16-alpine
    container_name: practice01-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: practice_db
      PGDATA: /var/lib/postgresql/data/pgdata
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    command: postgres -c shared_buffers=128MB -c
max_connections=100 -c log_statement=all
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d practice_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
```

```
    container_name: practice01-pgadmin
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "8083:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  redis_data:
  mongodb_data:
  neo4j_data:
  neo4j_logs:
  es_data:
  postgres_data:
  pgadmin_data:
```

Запуск всех СУБД одной командой:

```
cd ~/practice01-databases/all-in-one
docker compose up -d
```

> ***Внимание**: для запуска всех пяти СУБД одновременно требуется минимум **8 ГБ оперативной памяти**. Если ресурсов недостаточно, запускайте СУБД по отдельности из соответствующих директорий.*

Сводная таблица портов

СУБД	Порт сервера	Веб-интерфейс	URL
Redis	6379	Redis Commander	http://localhost:8081
MongoDB	27017	Mongo Express	http://localhost:8082

Neo4j	7474, 7687	Neo4j Browser (встроенный)	http://localhost:7474
Elasticsearch	9200	Kibana	http://localhost:5601
PostgreSQL	5432	pgAdmin 4	http://localhost:8083

---

7. Контрольные вопросы

1. **Что такое Docker-контейнер и чем он отличается от виртуальной машины?**
2. **Для чего используется файл `docker-compose.yml`? Какие основные секции он содержит?**
3. **Зачем нужны тома (volumes) в Docker? Что произойдет с данными, если удалить контейнер без использования томов?**
4. **Что означает параметр `depends_on` в docker-compose? Гарантирует ли он, что зависимый сервис полностью готов к работе?**
5. **Назовите модель данных каждой из пяти СУБД. Для каких задач лучше подходит каждая из них?**
6. **Что такое ACID-транзакции? Какие из рассмотренных СУБД их поддерживают?**
7. **Чем отличается тип поля `text` от `keyword` в Elasticsearch? Приведите примеры использования каждого.**
8. **Что такое Cypher в Neo4j? Чем графовый запрос отличается от SQL-запроса при поиске связанных данных?**
9. **Почему Redis хранит данные в оперативной памяти? Какие механизмы персистентности он поддерживает?**
10. **Что такое `healthcheck` в Docker Compose? Почему его важно настраивать?**
11. **Объясните разницу между командами `docker compose stop` и `docker compose down`. Когда следует использовать каждую?**

12. **Какие стратегии перезапуска (restart policy) существуют в Docker? Чем `unless-stopped` отличается от `always`?**
13. **Зачем ограничивать потребление памяти для Elasticsearch и Neo4j (параметры JVM heap)? Что произойдет, если этого не сделать?**
14. **Сравните веб-интерфейсы всех пяти СУБД. Какой из них, по вашему мнению, наиболее функциональный и почему?**
15. **Какие потенциальные проблемы безопасности существуют в представленных конфигурациях? Что нужно изменить для production-окружения?**

---

8. Задания для самостоятельной работы

1. Измените пароли во всех конфигурациях и убедитесь, что все сервисы продолжают работать корректно.
2. Для PostgreSQL: создайте скрипт инициализации `.sql`, который автоматически создает таблицы и вставляет тестовые данные при первом запуске контейнера. Подключите его через volume в директорию `/docker-entrypoint-initdb.d`.
3. Для MongoDB: создайте коллекцию с вложенными документами (например, заказы с товарами) и выполните агрегационный запрос для подсчета суммы заказов.
4. Для Redis: реализуйте простой счетчик посещений с помощью команды `INCR`. Установите TTL на ключ и проследите за его удалением.
5. Для Neo4j: создайте граф социальной сети (пользователи и связи «дружбы»). Напишите Cypher-запрос для поиска «друзей друзей» с исключением уже существующих друзей.
6. Для Elasticsearch: проиндексируйте набор текстовых документов (например, описания книг) и выполните полнотекстовый поиск с ранжированием результатов.

Составлено для практических занятий по дисциплине «Базы данных».